Documento de Arquitectura — Modelo C4

ServIA — Asistente de Inteligencia Artificial para el Sector Público

Nombre del proyecto: ServIA — Asistente de Inteligencia Artificial para el Sector Público
Institución que conduce el proyecto: MICITT — Dirección de Gobernanza
Fecha: setiembre 2026
Versión del documento: 1.1

1. Introducción

El modelo C4, propuesto por Simon Brown, describe la arquitectura de un sistema de software mediante cuatro niveles de abstracción que se recorren de lo general a lo particular: Contexto, Contenedores, Componentes y Código. Cada nivel amplía un elemento del nivel anterior, de modo que el lector puede detenerse en el grado de detalle que necesita.

Este documento desarrolla los tres primeros niveles para ServIA. El nivel de Código no se elabora en esta versión: el detalle de las clases se deriva del Modelo de Dominio y del código fuente, y cambia con mayor frecuencia que los niveles superiores.

Como el resto de la documentación del proyecto, los diagramas describen la arquitectura planificada del sistema. Toman como punto de partida la arquitectura ya implementada y señalan en amarillo los ajustes pendientes que el ERS identifica en sus párrafos «Estado actual», para que el desarrollo pueda ubicar en qué contenedor o componente corresponde implementar cada uno (sección 8).

El código PlantUML de cada diagrama se encuentra en la carpeta Diagramas/Arquitectura_C4, junto a este documento. Utiliza la biblioteca C4-PlantUML que PlantUML ya incluye (instrucción !include <C4/...>), por lo que no requiere descargar archivos adicionales para generarse, y el motor de acomodo ELK (instrucción !pragma layout elk), que traza las relaciones con líneas rectas en ángulo recto para que los flujos se sigan con facilidad.

2. Objetivo del documento

El objetivo de este documento es ofrecer una vista común de la arquitectura de ServIA para su desarrollo y su evaluación: qué personas y sistemas interactúan con él, en qué contenedores se ejecuta, qué responsabilidades asume cada componente del servidor y cómo se comunican entre sí.

Además, relaciona cada componente con los requisitos funcionales del ERS que implementa. De esta forma complementa los casos de uso, el modelo de dominio y los procesos con la perspectiva técnica: dónde se implementa cada funcionalidad y qué tecnología la soporta.

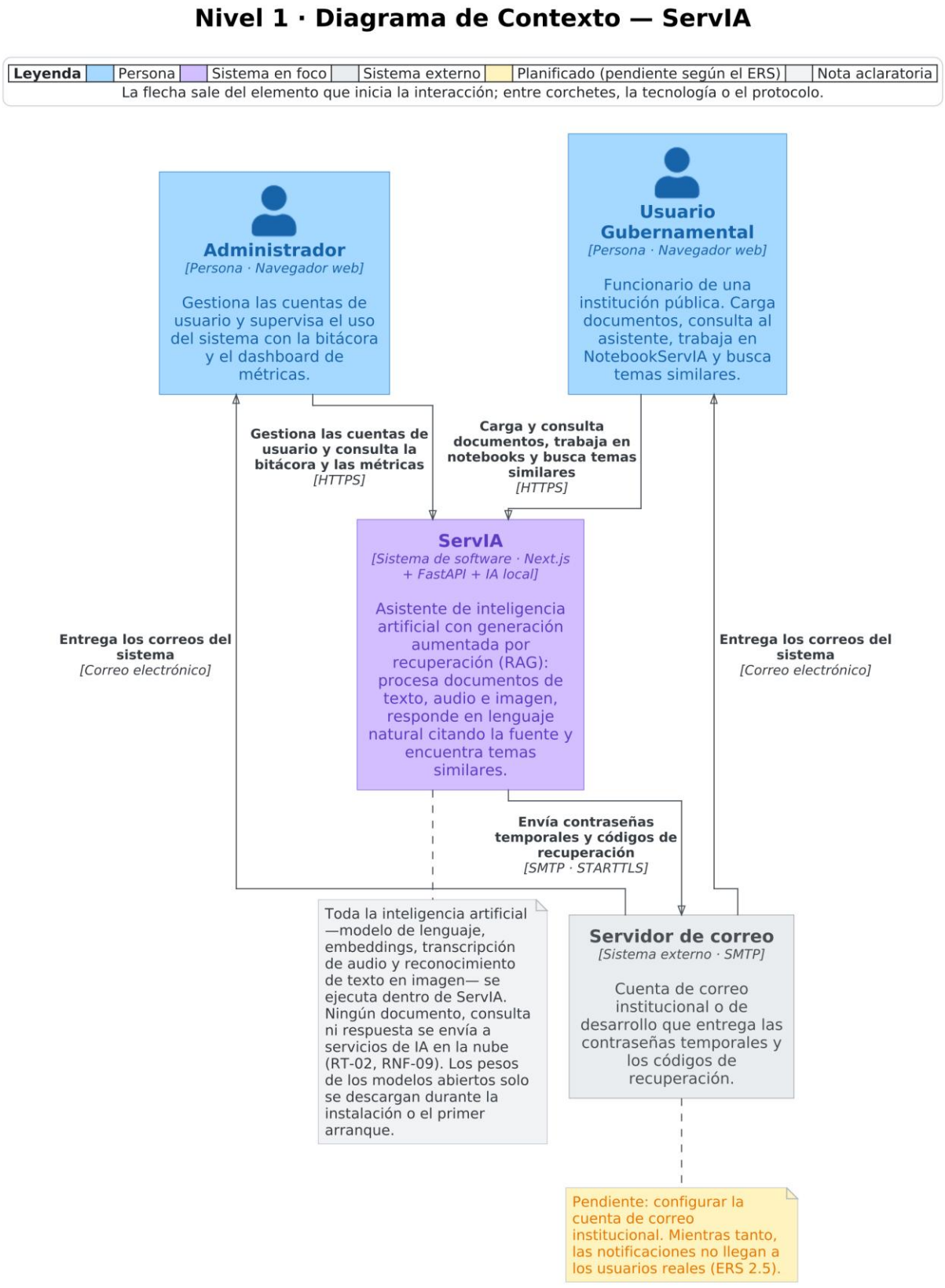
3. Convenciones de los diagramas

Los cuatro diagramas usan la misma notación, basada en los colores del machote de referencia:

Elemento	Representación	Significado
Persona	Rectángulo azul con ícono de persona	Rol humano que usa el sistema: Usuario Gubernamental o Administrador.
Sistema, contenedor o componente propio	Rectángulo morado	Elemento de ServIA que se describe en el nivel correspondiente.
Base de datos o almacenamiento	Cilindro celeste	Contenedor que conserva datos: base relacional, base vectorial, almacén en memoria o archivos.
Infraestructura compartida	Rectángulo verde agua, dentro del recuadro «Núcleo»	Componente del que dependen varios módulos del mismo contenedor (nivel 3).
Sistema externo o contenedor fuera de foco	Rectángulo gris	Elemento fuera del límite del sistema o del contenedor que se describe.
Límite	Recuadro con borde discontinuo	Frontera del sistema (nivel 2) o del contenedor (nivel 3).
Nota planificada	Nota amarilla	Ajuste pendiente según el ERS.
Nota aclaratoria	Nota gris	Información complementaria sobre un elemento.
Relación	Flecha con texto y [tecnología]	Sale del elemento que inicia la interacción; el texto indica qué se hace y los corchetes, el protocolo o mecanismo.

4. Nivel 1 — Diagrama de Contexto

Muestra ServIA como una sola caja, junto con las personas que lo usan y el único sistema externo con el que se comunica.



4.1 Elementos del diagrama

Elemento	Tipo	Descripción
Usuario Gubernamental	Persona	Funcionario de una institución pública. Carga documentos, consulta al asistente, trabaja en NotebookServIA y busca temas similares.
Administrador	Persona	Gestiona las cuentas de usuario y supervisa el uso del sistema con la bitácora y el dashboard de métricas.
ServIA	Sistema de software (en foco)	Asistente de inteligencia artificial con generación aumentada por recuperación (RAG): procesa documentos de texto, audio e imagen, responde en lenguaje natural citando la fuente y encuentra temas similares.
Servidor de correo	Sistema externo	Cuenta de correo institucional o de desarrollo que entrega las contraseñas temporales y los códigos de recuperación.

4.2 Relaciones

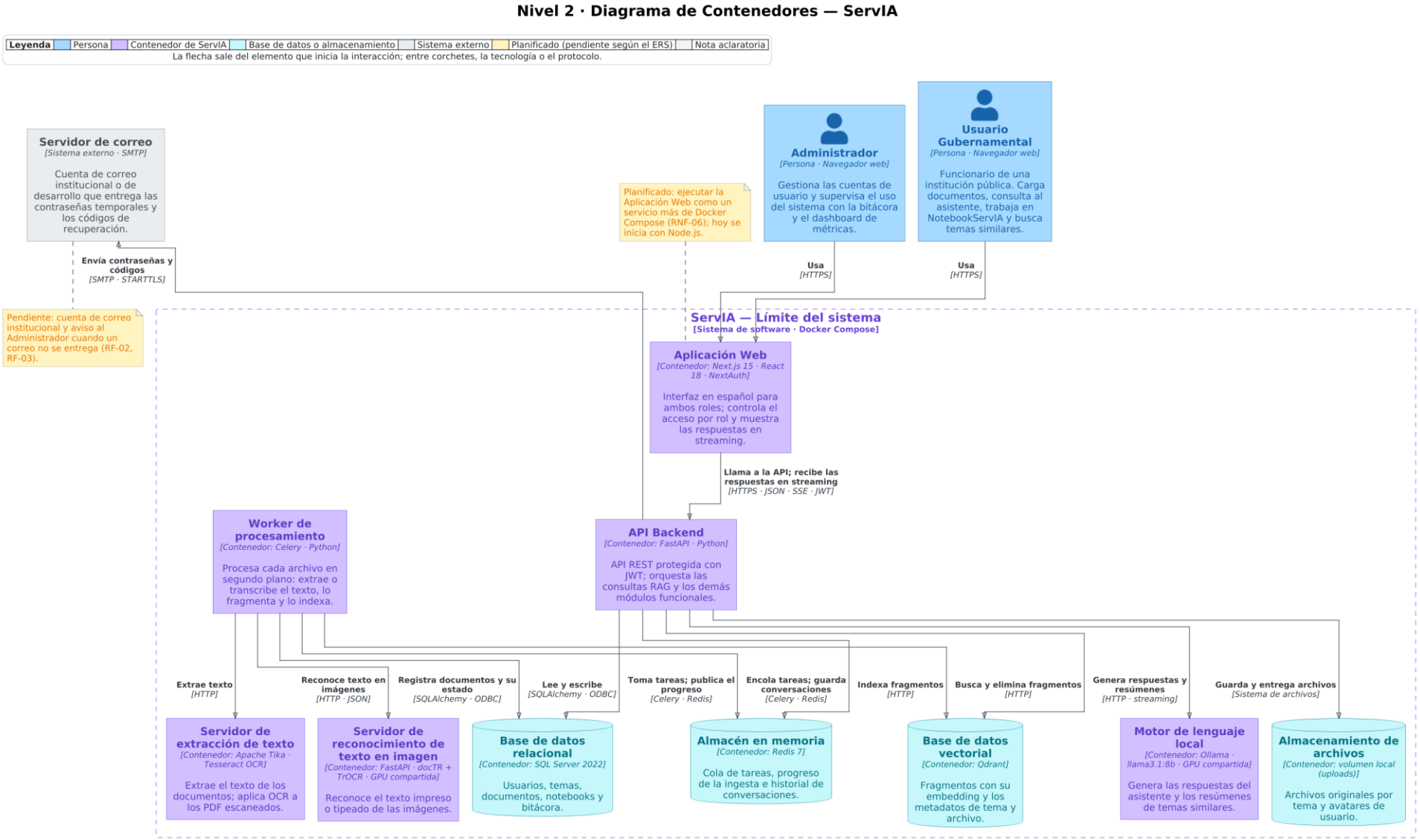
Origen	Destino	Interacción	Tecnología
Usuario Gubernamental	ServIA	Carga documentos, consulta al asistente, trabaja en NotebookServIA y busca temas similares	HTTPS
Administrador	ServIA	Gestiona las cuentas de usuario y consulta la bitácora y las métricas	HTTPS
ServIA	Servidor de correo	Envía contraseñas temporales y códigos de recuperación	SMTP · STARTTLS
Servidor de correo	Usuario Gubernamental	Entrega los correos del sistema	Correo electrónico
Servidor de correo	Administrador	Entrega los correos del sistema	Correo electrónico

El diagrama refleja una decisión central del proyecto: el único sistema externo es el servidor de correo. El modelo de lenguaje, los embeddings, la transcripción de audio y el reconocimiento de texto en imagen se ejecutan dentro de ServIA (RT-02, RNF-09), por lo que no aparecen como sistemas externos. Los pesos de los modelos abiertos se descargan una única vez, durante la instalación o el primer arranque, y no forman parte del uso normal del sistema.

La comunicación entre el navegador y ServIA se representa con HTTPS, el protocolo que se exige en cualquier entorno expuesto más allá del desarrollo local (RNF-08); en el entorno de desarrollo, los servicios se acceden por HTTP en el propio equipo.

5. Nivel 2 — Diagrama de Contenedores

Abre la caja de ServIA y muestra sus contenedores: cada aplicación o almacén de datos que se ejecuta por separado.



ServIA es un monolito modular: la lógica propia se concentra en la API Backend y en el Worker de procesamiento, que comparten el mismo código fuente y se ejecutan como procesos distintos. Los ocho servicios de Docker Compose descritos en el ERS (2.1) aparecen como contenedores, junto con la Aplicación Web y el almacenamiento de archivos, un volumen local montado en ambos procesos.

5.1 Contenedores

Contenedor	Tecnología	Responsabilidad	Ejecución
Aplicación Web	Next.js 15, React 18, NextAuth, HeroUI, Chart.js	Interfaz en español para ambos roles: inicio de sesión, Asistente Virtual, NotebookServIA, Ingesta de Datos, Temas Similares y administración. Controla el acceso a cada página según el rol, renueva la sesión mientras hay actividad y muestra las respuestas del asistente a medida que se generan.	Node.js (npm), puerto 3000. Cuenta con Dockerfile propio.
API Backend	Python, FastAPI, Uvicorn, SQLAlchemy, LangChain, sentence-transformers	Expone la API REST protegida con JWT para todos los módulos: autenticación, usuarios, ingesta, consultas RAG en streaming, NotebookServIA, temas similares y bitácora. Genera el embedding de cada consulta y registra la auditoría de las acciones.	Servicio backend, puerto 8000. Aplica las migraciones de Alembic al iniciar.
Worker de procesamiento	Python, Celery, faster-whisper, sentence-transformers	Procesa en segundo plano cada archivo cargado: obtiene su texto (extracción, transcripción de audio o reconocimiento en imagen), lo fragmenta, genera los embeddings, los indexa y actualiza el estado del documento.	Servicio celery-worker. Una tarea a la vez, con un máximo de 2 horas por tarea.
Servidor de extracción de texto	Apache Tika Server, Tesseract OCR	Extrae el texto de PDF, DOC, DOCX, RTF y HTML, y aplica OCR a los PDF escaneados.	Servicio tika, puerto 9998.
Servidor de reconocimiento de texto en imagen	Python, FastAPI, docTR, TrOCR, PyTorch (GPU)	Localiza las líneas de texto de una imagen (docTR) y reconoce su contenido (TrOCR). Está orientado a texto impreso o tipeado; la escritura a mano es una limitación aceptada (RF-10).	Servicio servidor-htr, puerto 9100. Libera la GPU tras 120 s sin uso.
Motor de lenguaje local	Ollama, modelo llama3.1:8b (GPU)	Genera las respuestas del asistente y los resúmenes de temas similares con el modelo llama3.1:8b, sin salir de la infraestructura local.	Servicio ollama, puerto 11434. Libera la GPU tras 2 min sin uso y atiende una generación a la vez.
Base de datos relacional	Microsoft SQL Server 2022; esquema versionado con Alembic	Guarda usuarios y roles, temas, documentos y su estado de procesamiento, notebooks y la bitácora.	Servicio sqlserver, puerto 1433, con volumen persistente.
Base de datos vectorial	Qdrant	Guarda los fragmentos de texto con su embedding de 1024 dimensiones y los metadatos de tema y archivo que permiten citar la fuente y acotar las búsquedas.	Servicio qdrant, puertos 6333 (REST) y 6334 (gRPC), con volumen persistente.

Contenedor	Tecnología	Responsabilidad	Ejecución
Almacén en memoria	Redis 7	Funciona como cola de tareas de Celery (base 0) y almacén de sus resultados (base 1), guarda el progreso de cada tarea de ingesta y conserva el historial de conversaciones durante 30 días (base 2).	Servicio redis, puerto 6379, con un límite de 1 GB.
Almacenamiento de archivos	Sistema de archivos local (carpeta uploads)	Conserva los archivos originales cargados, organizados por tema, y las fotografías de perfil de los usuarios.	Volumen montado en los servicios backend y celery-worker.

5.2 Relaciones

Origen	Destino	Interacción	Tecnología
Usuario Gubernamental	Aplicación Web	Usa	HTTPS
Administrador	Aplicación Web	Usa	HTTPS
Aplicación Web	API Backend	Llama a la API REST con el token JWT y recibe las respuestas del asistente en streaming	HTTPS · JSON · SSE · JWT
API Backend	Servidor de correo	Envía las contraseñas temporales y los códigos de recuperación	SMTP · STARTTLS
API Backend	Motor de lenguaje local	Genera respuestas y resúmenes	HTTP · streaming
API Backend	Almacenamiento de archivos	Guarda los archivos cargados y los avatares; entrega y elimina los originales	Sistema de archivos
API Backend	Base de datos relacional	Lee y escribe	SQLAlchemy · ODBC
API Backend	Almacén en memoria	Encola una tarea por archivo, consulta su progreso y guarda las conversaciones	Celery · Redis
API Backend	Base de datos vectorial	Busca los fragmentos relevantes y elimina los de un documento	HTTP
Worker de procesamiento	Almacén en memoria	Toma las tareas de la cola y publica su progreso	Celery · Redis
Worker de procesamiento	Base de datos relacional	Registra el documento, su estado de procesamiento y la bitácora de la ingesta	SQLAlchemy · ODBC
Worker de procesamiento	Base de datos vectorial	Indexa los fragmentos con sus embeddings y metadatos	HTTP
Worker de procesamiento	Servidor de extracción de texto	Envía los documentos de texto para extraer su contenido	HTTP
Worker de procesamiento	Servidor de reconocimiento de texto en imagen	Envía las imágenes para reconocer su texto	HTTP · JSON

El diagrama muestra dos flujos independientes. El flujo interactivo va de la Aplicación Web a la API Backend, que consulta las bases de datos y el motor de lenguaje y responde en streaming. El flujo de ingesta es asíncrono: la API guarda el archivo, encola una tarea en Redis y responde de inmediato (RNF-02); el Worker toma la tarea, obtiene el texto con Tika, con el servidor de reconocimiento de texto en imagen o con su transcriptor de audio, lo indexa en Qdrant y actualiza el estado del documento en SQL Server.

La entrega de los correos a las personas se muestra en el nivel 1; en este nivel interesa qué contenedor se comunica con el servidor de correo, que es la API Backend.

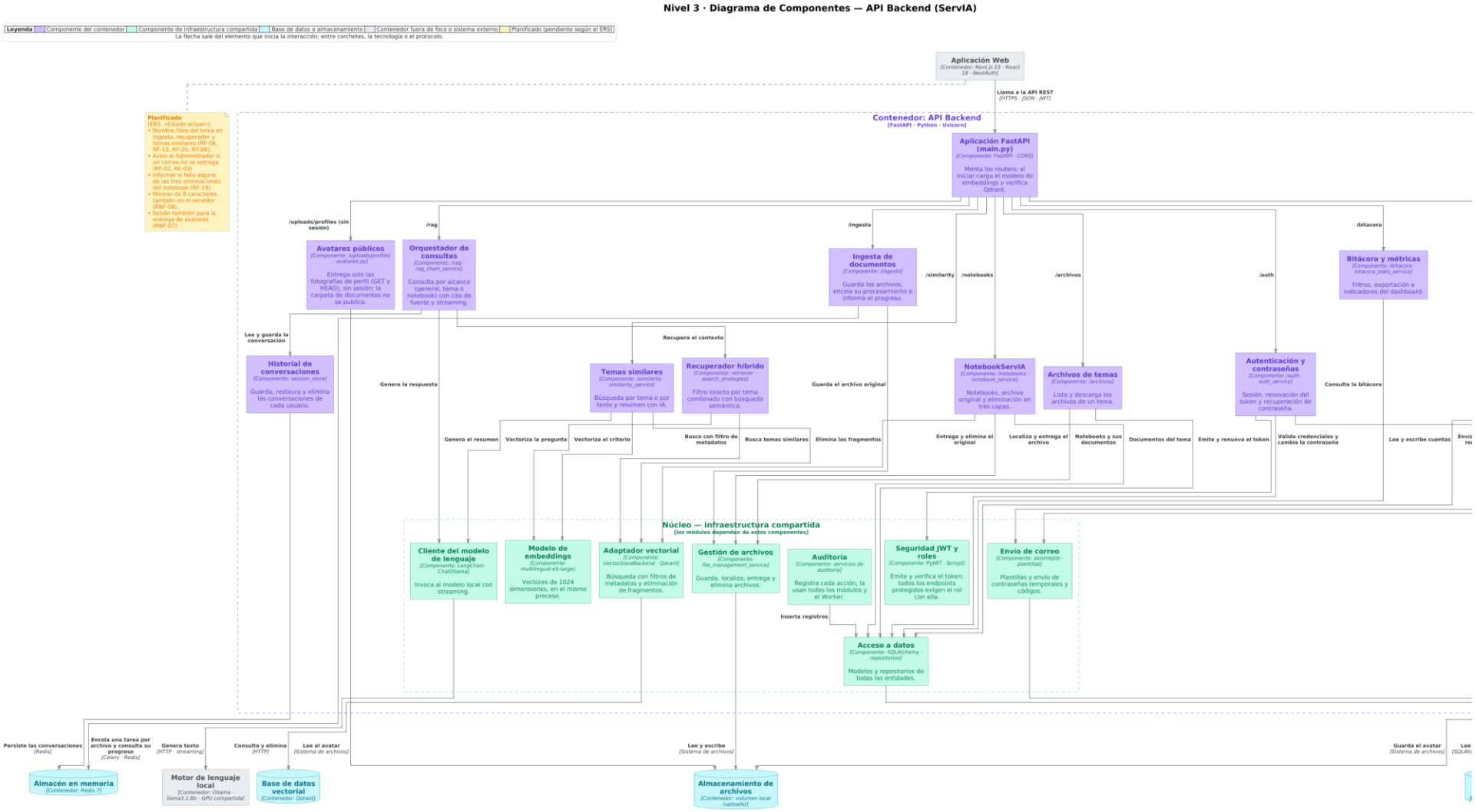
6. Nivel 3 — Diagramas de Componentes

El nivel de componentes se desarrolla para los dos contenedores que concentran la lógica propia de ServIA en el servidor: la API Backend y el Worker de procesamiento. Los demás contenedores son productos de terceros que se usan sin modificar (SQL Server, Qdrant, Redis, Ollama y Apache Tika), un servicio de una sola responsabilidad descrito en RF-09 (el servidor de reconocimiento de texto en imagen) o la capa de presentación, cuyas pantallas y comportamiento se describen en los casos de uso.

En ambos diagramas, los componentes de infraestructura compartida se agrupan en el recuadro «Núcleo»: ningún módulo accede directamente a SQL Server, Qdrant, Ollama o al sistema de archivos, sino a través del componente correspondiente. Las rutas de la columna «Ubicación en el código» son relativas a la carpeta backend/app, salvo que se indique otra.

6.1 Componentes de la API Backend

Cada módulo funcional expone sus endpoints con un router de FastAPI y delega en el núcleo el acceso a datos, a la base vectorial, al modelo de lenguaje y a los archivos.



La tabla siguiente describe cada componente de la API Backend, su ubicación en el código y los requisitos funcionales que implementa. Seguridad JWT y roles y Auditoría no se conectan con flechas a cada módulo para no saturar el diagrama: todos los endpoints protegidos usan la primera como dependencia (RF-02, RNF-07) y todos los módulos registran sus acciones mediante la segunda (RF-21).

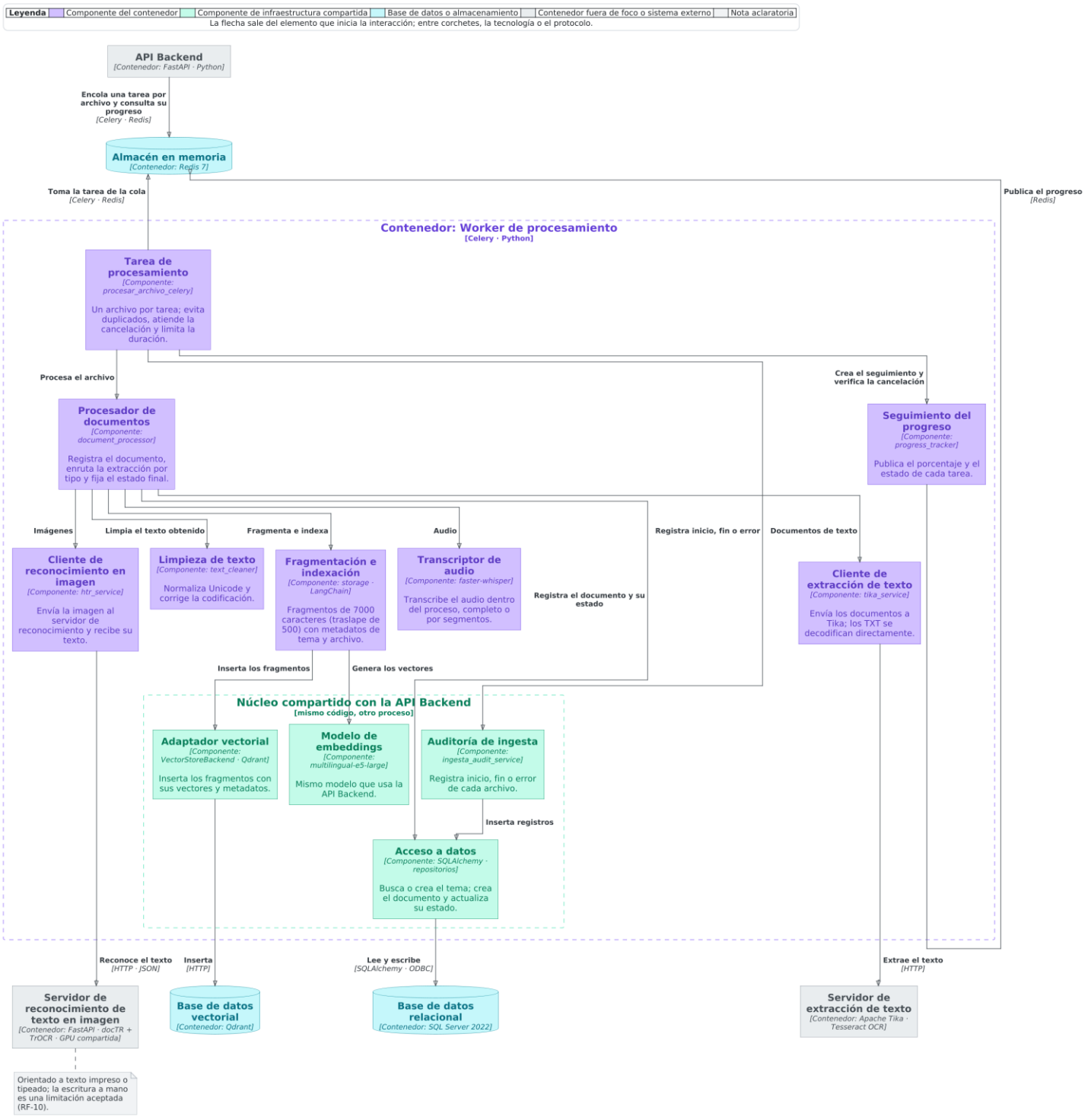
Componente	Ubicación en el código	Responsabilidad	RF
Aplicación FastAPI (main.py)	backend/main.py	Punto de entrada de la API: monta los routers de cada módulo, incluida la entrega pública de avatares, configura CORS, y al iniciar carga el modelo de embeddings y verifica la conexión con Qdrant (si algo falla, la API no arranca).	—
Autenticación y contraseñas	routes/auth.py · services/ auth_service.py	Inicio y cierre de sesión, renovación del token, cambio voluntario de contraseña y recuperación por código de 6 dígitos.	RF-01, RF-03, RF-04
Usuarios y perfil	routes/usuarios.py · services/usuario_service.py · services/avatar_service.py	Creación, edición y desactivación lógica de cuentas; restablecimiento de la contraseña temporal por el Administrador; carga, selección y restablecimiento del avatar.	RF-02, RF-03, RF-04
Ingesta de documentos	routes/ingesta.py	Recibe los archivos con su tema, guarda cada original, encola una tarea de procesamiento por archivo, informa el progreso de cada tarea y permite cancelarla.	RF-05, RF-06, RF-07
Orquestador de consultas	routes/rag.py · services/ RAG/rag_chain_service.py · services/RAG/prompts/	Resuelve cada consulta según su alcance (general, tema o notebook): recupera el contexto, arma el prompt con las reglas de cita de fuente y sin invención, genera la respuesta en streaming y la agrega a la conversación.	RF-11, RF-12, RF-15, RF-16
Recuperador híbrido	services/RAG/retriever.py · services/RAG/search_ strategies.py	Recupera los fragmentos más relevantes dentro del alcance; si la pregunta menciona un tema, combina el filtro exacto por tema con la búsqueda semántica y reintenta sin filtro cuando el tema no tiene datos.	RF-11, RF-13
Historial de conversaciones	services/RAG/session_ store.py · services/RAG/ conversation_history_ redis.py	Guarda las conversaciones de cada usuario, las lista, las restaura con su contexto completo y las elimina; el modelo recibe solo los mensajes más recientes.	RF-14
NotebookServIA	routes/notebooks.py · services/notebook_ service.py	Crea y lista los notebooks del usuario, lista sus documentos, entrega el archivo original para la previsualización y elimina un documento de sus tres capas (fragmentos, archivo y registro), verificando siempre que el notebook pertenezca al usuario.	RF-17, RF-18, RF-19
Temas similares	routes/similarity.py · services/busqueda_ similares/	Compara un tema de referencia o un texto libre con la base vectorial, calcula y clasifica el porcentaje de similitud de hasta 30 resultados y genera bajo demanda el resumen con IA de un resultado.	RF-20

Componente	Ubicación en el código	Responsabilidad	RF
Archivos de temas	routes/archivos.py	Lista y descarga los archivos originales de un tema, incluidos los que cita el asistente en sus respuestas.	RF-12, RF-20
Avatares públicos	routes/avatares.py	Entrega solo las fotografías de perfil por GET y HEAD en /uploads/profiles/{nombre}, sin sesión y con Vary: Origin. Es una excepción deliberada a «todo endpoint exige rol» (RF-04.20, RNF-07.22); la carpeta de documentos no se publica.	RF-04, RNF-07
Bitácora y métricas	routes/bitacora.py · services/bitacora/bitacora_service.py · services/bitacora/bitacora_stats_service.py	Consulta la bitácora con filtros combinables, registra su exportación y calcula los indicadores y gráficos del dashboard de métricas.	RF-22, RF-23
Seguridad JWT y roles (núcleo)	auth/jwt_auth.py · repositories/usuario_repository.py	Emite y verifica el token JWT y exige el rol correspondiente en cada endpoint protegido, que la declara como dependencia. Las contraseñas se comparan contra su hash bcrypt.	RF-01, RF-02
Auditoría (núcleo)	services/bitacora/ (un servicio de auditoría por módulo)	Registra de forma automática cada acción relevante en la bitácora; la invocan todos los módulos funcionales y el Worker. Si el registro falla, la acción del usuario se completa de todas formas.	RF-21
Envío de correo (núcleo)	email/ · routes/email.py (verificación de la configuración)	Arma las plantillas de los correos y envía las contraseñas temporales y los códigos de recuperación por SMTP.	RF-02, RF-03
Acceso a datos (núcleo)	db/ (modelos y sesión) · repositories/ · migraciones de Alembic	Modelos, sesión y repositorios de usuarios, temas, documentos, notebooks y bitácora. Es el único punto de acceso a SQL Server.	—
Modelo de embeddings (núcleo)	embeddings/embeddings.py	Convierte las consultas y los textos de referencia en vectores de 1024 dimensiones, dentro del mismo proceso de la API.	RF-08, RF-11
Adaptador vectorial (núcleo)	vectorstore/base.py · vectorstore/qdrant_backend.py	Interfaz única hacia la base vectorial: búsqueda por vector con filtros de metadatos, obtención de los fragmentos de un documento o tema y eliminación de fragmentos.	RF-08, RF-13, RF-19
Cliente del modelo de lenguaje (núcleo)	llm/llm_service.py	Invoca al modelo de lenguaje local con streaming y con los parámetros de generación configurados (temperatura, contexto y longitud de la respuesta).	RF-11, RF-20
Gestión de archivos (núcleo)	services/documentos/file_management_service.py	Guarda los archivos cargados en la carpeta de su tema y los localiza, entrega y elimina cuando otro módulo lo solicita.	RF-05, RF-18, RF-19

6.2 Componentes del Worker de procesamiento

El Worker ejecuta el pipeline de ingesta de un archivo por tarea: toma la tarea de la cola, obtiene el texto según el tipo de archivo, lo fragmenta y lo indexa.

Nivel 3 · Diagrama de Componentes — Worker de procesamiento (ServIA)



El procesador de documentos elige la vía de extracción según la extensión del archivo: el audio va al transcriptor, las imágenes al cliente de reconocimiento —antes que a Tika, que las procesaría con OCR de texto impreso— y el resto de los formatos a Tika. Los componentes del núcleo son el mismo código que usa la API Backend, lo que garantiza que documentos y consultas se vectoricen con el mismo modelo (RF-08).

Componente	Ubicación en el código	Responsabilidad	RF
Tarea de procesamiento	celery_app.py (raíz de backend) · services/ingesta/async_processing/celery_tasks.py	Procesa un archivo por tarea. Antes de empezar verifica si un reintento ya lo había procesado, para no duplicarlo; revisa la cancelación durante el procesamiento y limita la duración a 2 horas.	RF-07
Seguimiento del progreso	services/ingesta/async_processing/progress_tracker.py	Publica el porcentaje, la etapa y el estado de cada tarea, que la API Backend consulta periódicamente para mostrarlos al usuario.	RF-07
Procesador de documentos	services/ingesta/file_management/document_processor.py	Valida el archivo, registra el documento como Pendiente, elige la vía de extracción según su tipo, limpia el texto, lo envía a indexar y deja el documento en Procesado o en Error con el detalle de la causa.	RF-05, RF-06, RF-07
Cliente de extracción de texto	services/ingesta/tika_service.py	Envía los documentos de texto a Apache Tika, con OCR habilitado para los PDF escaneados; los archivos TXT se decodifican directamente detectando su codificación.	RF-07
Transcriptor de audio	services/ingesta/audio_transcription/	Transcribe los archivos MP3, WAV, OGG y M4A dentro del proceso del Worker, de forma directa o por segmentos cuando el archivo es grande.	RF-07
Cliente de reconocimiento en imagen	services/ingesta/htr_service.py	Envía cada imagen al servidor de reconocimiento de texto en imagen y recibe el texto reconocido. Se usa antes que Tika para las imágenes.	RF-09, RF-10
Limpieza de texto	services/ingesta/file_management/text_cleaner.py	Normaliza el texto Unicode y corrige los problemas de codificación del texto obtenido antes de indexarlo.	RF-07
Fragmentación e indexación	vectorstore/storage.py	Divide el texto en fragmentos de 7000 caracteres con un traslape de 500, agrega a cada uno los metadatos de tema y archivo, y los envía a vectorizar e indexar.	RF-08
Modelo de embeddings (núcleo)	embeddings/embeddings.py	Genera el embedding de cada fragmento con el mismo modelo que usa la API Backend para las consultas.	RF-08
Adaptador vectorial (núcleo)	vectorstore/qdrant_backend.py	Inserta los fragmentos en Qdrant con sus vectores y metadatos.	RF-08
Acceso a datos (núcleo)	db/ · repositories/ · servicio de temas (conserva el nombre heredado, RT-06)	Busca o crea el tema indicado, crea el registro del documento y actualiza su ruta y su estado de procesamiento.	RF-06, RF-07
Auditoría de ingesta (núcleo)	services/bitacora/ingesta_audit_service.py	Registra en la bitácora el inicio, la finalización o el error del procesamiento de cada archivo.	RF-21

7. Decisiones de arquitectura

Las siguientes decisiones explican la forma de los diagramas y deben mantenerse al desarrollar los ajustes planificados:

ID	Decisión	Justificación	Requisitos
DA-01	Monolito modular	Un único código fuente organizado por módulos (router, servicios y repositorios) es más sostenible para un equipo de un solo desarrollador que un conjunto de microservicios. La separación entre la API Backend y el Worker aporta el aislamiento necesario para el procesamiento pesado.	ERS 2.1, RA-01
DA-02	Infraestructura 100 % local	La información institucional es sensible: el modelo de lenguaje, los embeddings, la transcripción y el reconocimiento de texto en imagen se ejecutan en contenedores propios, sin servicios de IA en la nube.	RT-02, RNF-09
DA-03	Ingesta asíncrona con cola de tareas	La carga responde en segundos aunque el procesamiento tarde minutos. El Worker procesa una tarea a la vez y detecta los reintentos para no duplicar documentos.	RNF-02, RF-07
DA-04	Respuestas en streaming	El texto se muestra a medida que el modelo lo genera, lo que reduce la espera percibida; la interfaz libera el envío si pasan 30 segundos sin recibir datos.	RF-11, RF-16
DA-05	GPU compartida con liberación por inactividad	El motor de lenguaje (5,5 GB) y el reconocimiento de texto en imagen (2,1 GB) ocupan 7,6 de los 8 GB de VRAM. Cada uno libera la memoria tras 2 minutos sin uso, y el motor atiende una generación a la vez para no dividir su contexto.	RT-03
DA-06	Base vectorial detrás de una interfaz	Los módulos usan VectorStoreBackend y no Qdrant directamente; cambiar de motor vectorial requiere solo otra implementación de la interfaz.	RF-08
DA-07	Un mismo modelo de embeddings en la API y el Worker	Los documentos y las consultas se vectorizan con el mismo modelo, de modo que sus vectores son comparables.	RF-08, RF-11
DA-08	Reconocimiento de imagen en un servicio propio	Aísla del backend las dependencias pesadas (PyTorch, modelos de visión, GPU) y sigue el mismo patrón que Tika: un contenedor consumido por HTTP.	RF-09, RT-03

8. Elementos planificados

Los ajustes siguientes provienen de los párrafos «Estado actual» del ERS y de las dependencias del proyecto. Se indican en amarillo en los diagramas y señalan el contenedor o componente donde corresponde implementarlos:

Contenedor o componente	Ajuste planificado	Requisito
Aplicación Web — Ingesta de Datos	Admitir imágenes y mostrar la advertencia sobre la escritura a mano.	RF-05, RF-10
Aplicación Web y API Backend — Ingesta de documentos	Indicar el tema por un nombre libre en lugar del identificador heredado del proyecto JusticIA.	RF-06, RT-06
API Backend — Recuperador híbrido	Detectar el tema mencionado en la pregunta por su nombre.	RF-13, RT-06
API Backend — Temas similares; Aplicación Web	Buscar por nombre de tema y mostrar la opción del menú como «Temas Similares».	RF-20, RT-06
Aplicación Web — NotebookServIA	Hacer efectiva la acción de reintentar en el chat del notebook.	RF-15
Aplicación Web y API Backend — NotebookServIA	Solicitar confirmación antes de eliminar un documento e informar si alguna de las tres eliminaciones falla.	RF-19
API Backend — Usuarios y perfil; Envío de correo	Avisar al Administrador cuando el correo con la contraseña temporal no se entregue.	RF-02, RF-03
Servidor de correo (sistema externo)	Configurar la cuenta de correo institucional para las notificaciones del sistema.	RF-03; ERS 2.5
API Backend — Autenticación; Usuarios y perfil	Validar también en el servidor el mínimo de 8 caracteres de las contraseñas.	RNF-08
Aplicación Web (contenedor)	Ejecutarla como un servicio más de Docker Compose.	RNF-06